



HEART_ MAKER/BREAKER

GROUP PROJECT

AASD 4001 TERM PROJECT



INTRODUCE
DATASET

APPROACH

SNIPPETS OF
EDA &
DATA
PREPROCESSING

TUNED MODELS

LOGISTIC
REGRESSION

DECISION TREE

RANDOM FOREST

SGD

SVM

FEATURE
ABLATION

REVIEW MODEL
PERFORMANCE

NEXT STEPS

01

02

03

04

INTRODUCE DATASET

10000 rows × 21 columns

APPROACH

SNIPPETS OF EDA & DATA PREPROCESSING

Numeric Features

```
['Age', 'Blood Pressure', 'Cholesterol  
Level', 'BMI', 'Sleep Hours',  
'Triglyceride Level', 'Fasting Blood  
Sugar', 'CRP Level', 'Homocysteine Level']
```

Categorical Features

```
['Gender', 'Exercise Habits', 'Smoking',  
'Family Heart Disease', 'Diabetes', 'High  
Blood Pressure', 'Low HDL Cholesterol',  
'High LDL Cholesterol', 'Alcohol  
Consumption', 'Stress Level', 'Sugar  
Consumption']
```

Label (Imbalanced)

```
Heart Disease Status:  
'No': 8000 (80.00%)  
'Yes': 2000 (20.00%)
```

Which ML approach is the best for our disease classification problem?

Approach:

Obtain understanding of dataset, EDA
Data preprocessing, drop, fill missing values, scaling
Split data, `stratify=y`
Tune with GridSearchCV, `class_weight='balanced'`
Assess tuned models, `scoring='recall'`
Check for overfitting, feature ablation

01



INTRODUCE DATASET

APPROACH

SNIPPETS OF EDA & DATA PREPROCESSING

Age	
count	9971.000000
mean	49.296259
std	18.193970
min	18.000000
25%	34.000000
50%	49.000000
75%	65.000000
max	80.000000

Exercise Habits:

'High': 3372 (33.72%)
'Medium': 3332 (33.32%)
'Low': 3271 (32.71%)
nan: 25 (0.25%)

```
df.isnull().sum()
```

✓ 0.0s

Age	29
Gender	19
Blood Pressure	19

Gender_Male	Smoking_Yes
1	1
0	0
1	0
0	1

```
# Fill NaN values in numeric columns  
using median  
for col in numeric_columns:  
    df[col] =  
df[col].fillna(df[col].median())
```

```
df = df.drop([  
    'Exercise Habits',  
    'Alcohol Consumption',  
    'Stress Level',  
    'Sleep Hours',  
    'Sugar Consumption'  
], axis=1)
```

01

TUNED MODELS

LOGISTIC REGRESSION

- Baseline model established
- ❖ Highlighted poor detection of the positive class.



Accuracy: 52%

DECISION TREE

RANDOM FOREST

SGD

SVM

- Applied hyperparameter tuning
- ❖ Used GridSearchCV
 - Cross-Validation and
 - Class_weight='balanced'



Class 0 → P: 0.80 | R: 0.54 | F1: 0.64

Class 1 → P: 0.20 | R: 0.46 | F1: 0.28

After Tuning

Accuracy: 54%

Class 0 → P: 0.80 | R: 0.54 | F1: 0.64

Class 1 → P: 0.20 | R: 0.46 | F1: 0.28

02



TUNED MODELS

LOGISTIC REGRESSION

DECISION TREE

RANDOM FOREST

SGD

SVM

→ Removed selected features

- BMI
- High Blood Pressure

→ Evaluated effect of feature reduction on model performance

→ Compared results before and after removal

After removing selected features

Removed Features: 'BMI', 'Blood Pressure_Yes'

Accuracy: 0.5

		precision	recall	f1-score	support
	0	0.79	0.51	0.62	1600
	1	0.19	0.46	0.27	400
accuracy				0.50	2000
macro avg		0.49	0.49	0.44	2000
weighted avg		0.67	0.50	0.55	2000

02

TUNED MODELS

LOGISTIC
REGRESSION

DECISION TREE

RANDOM FOREST

SGD

SVM

OPTIMAL HYPERPARAMETERS FOR DECISION TREE:

```
{'criterion': 'gini', 'max_depth': 7}
```

DECISION TREE TUNED TEST REPORT

Classification Report:

	precision	recall	f1-score	support
0	0.80	0.60	0.68	1600
1	0.20	0.40	0.26	400
accuracy			0.56	2000
macro avg	0.50	0.50	0.47	2000
weighted avg	0.68	0.56	0.60	2000

```
base_dt =  
DecisionTreeClassifier(random_state=42  
,class_weight='balanced')  
  
param_grid = {  
    'max_depth': [3, 5, 7, 10, None],  
    'criterion': ['gini', 'entropy']  
}  
  
grid_search = GridSearchCV(  
    estimator=base_dt,  
    param_grid=param_grid,  
    scoring='recall', # Aligned approach  
    cv=5,  
    n_jobs=-1  
)
```

02

TUNED MODELS

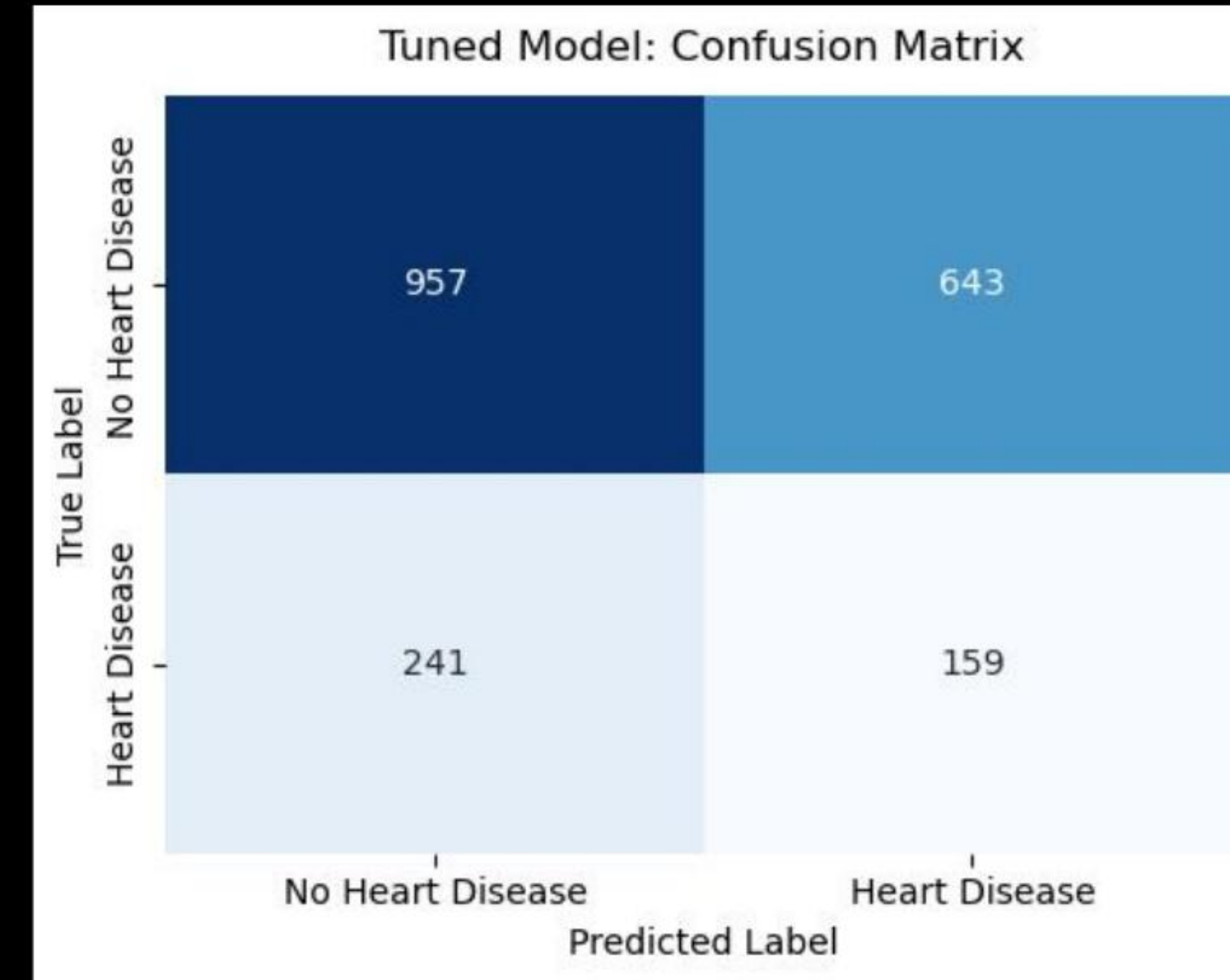
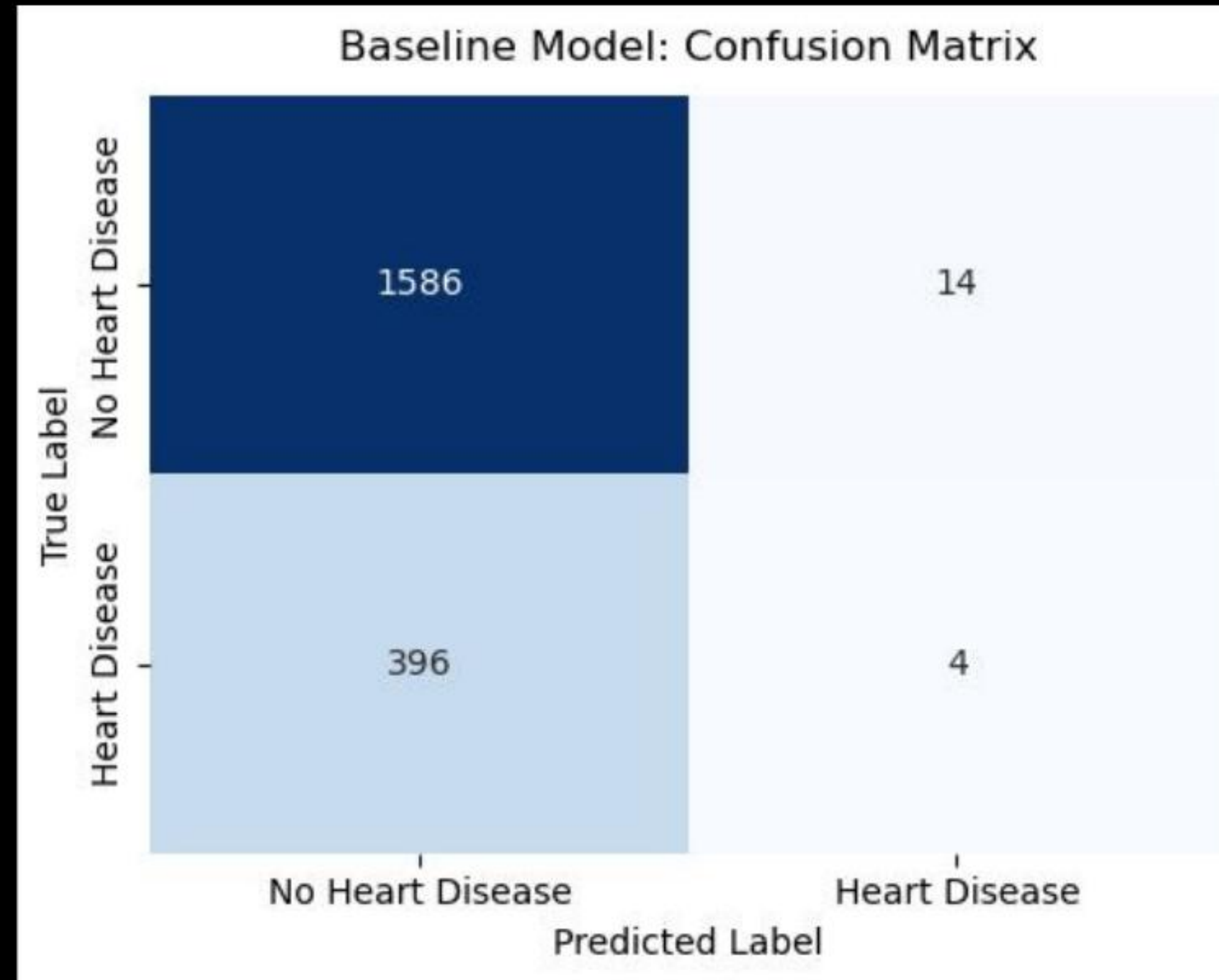
LOGISTIC
REGRESSION

DECISION TREE

RANDOM FOREST

SGD

SVM



02

U

TUNED MODELS

LOGISTIC
REGRESSION

DECISION TREE

RANDOM FOREST

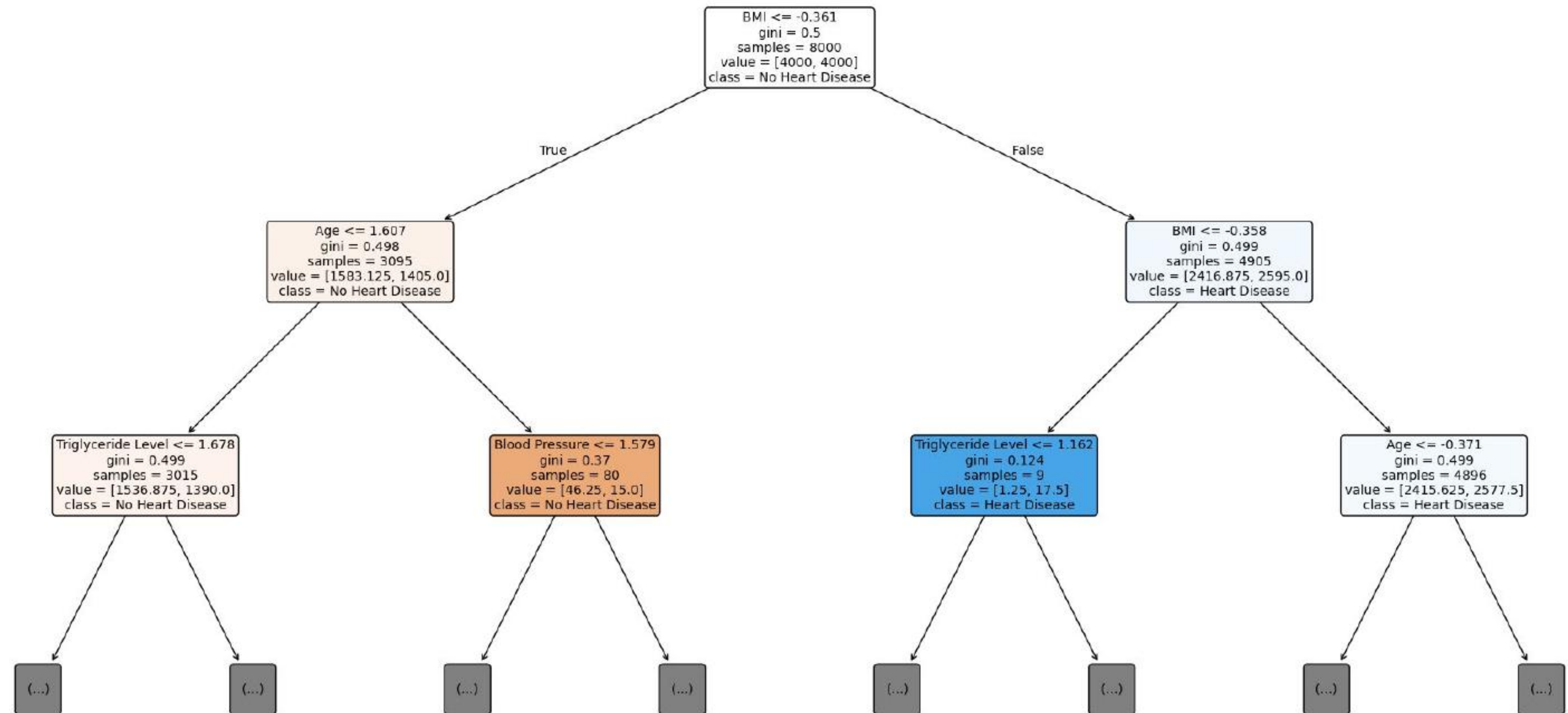
SGD

SVM

02



Tuned Decision Tree Diagnostic Splitting Rules (Top 2 Levels)



TUNED MODELS

LOGISTIC REGRESSION

DECISION TREE

RANDOM FOREST

SGD

SVM

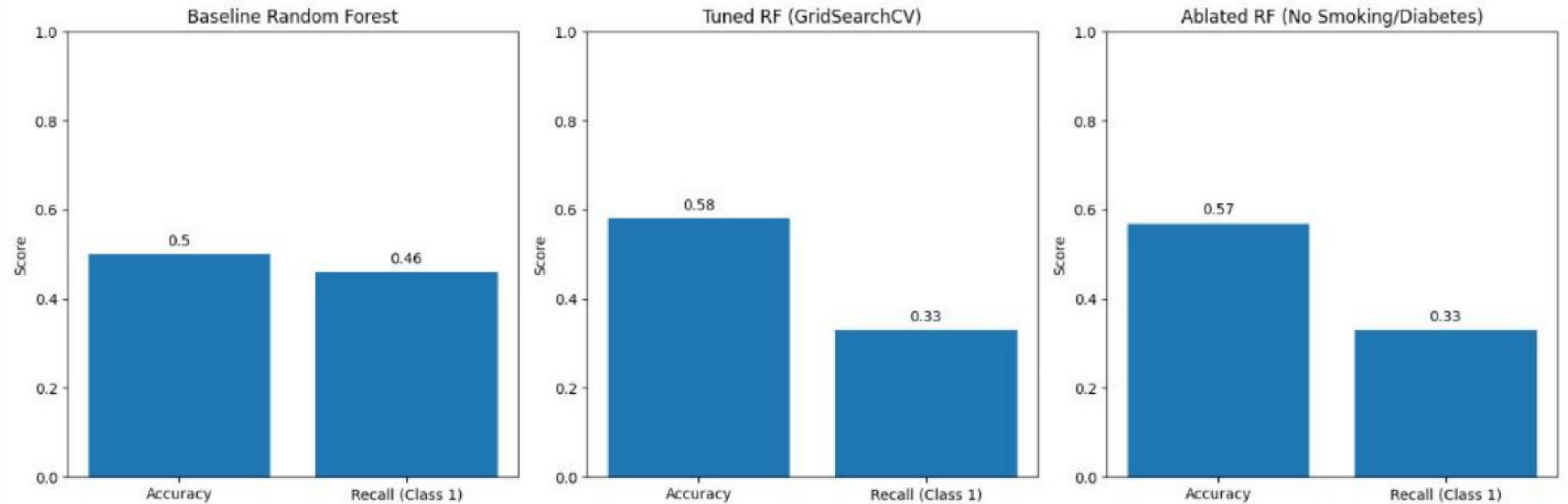
02



```
rf_clf = RandomForestClassifier(  
    random_state=42,  
    class_weight='balanced' )
```

```
OPTIMAL HYPERPARAMETERS FOR RANDOM  
FOREST: {'max_depth': 5,  
        'n_estimators': 50}
```

```
X_three_dropped =  
X.drop(['Smoking_Yes',  
        'Diabetes_Yes'],axis=1)
```



TUNED MODELS

LOGISTIC
REGRESSION

DECISION TREE

RANDOM FOREST

SGD

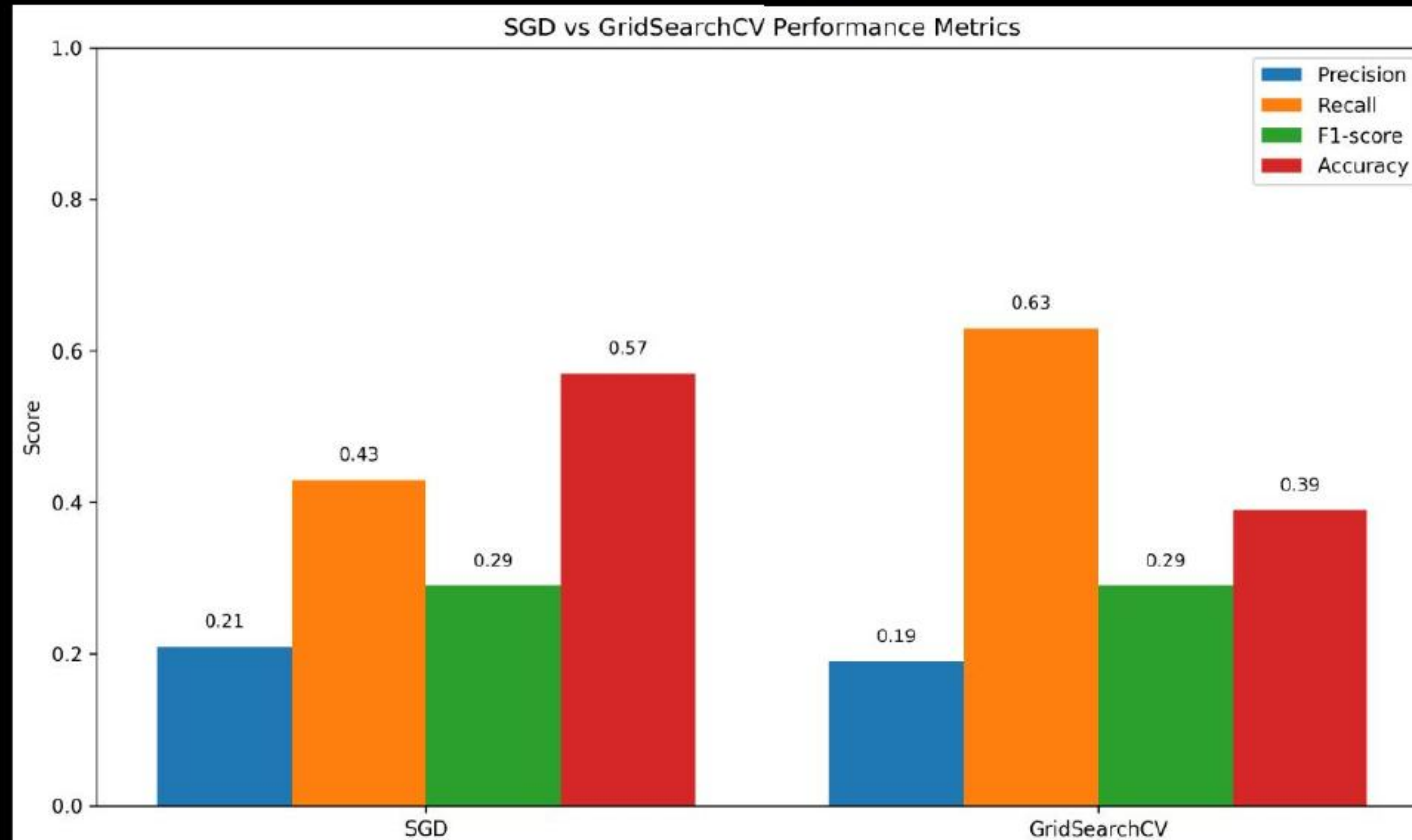
SVM

02



```
# Create SGD model
sgd = SGDClassifier(
    random_state=42,
    class_weight='balanced'
)
```

```
# Create GridSearchCV object
grid_search = GridSearchCV(
    sgd,
    param_grid,
    cv=5,
    scoring='recall'
)
```



TUNED MODELS

LOGISTIC
REGRESSION

DECISION TREE

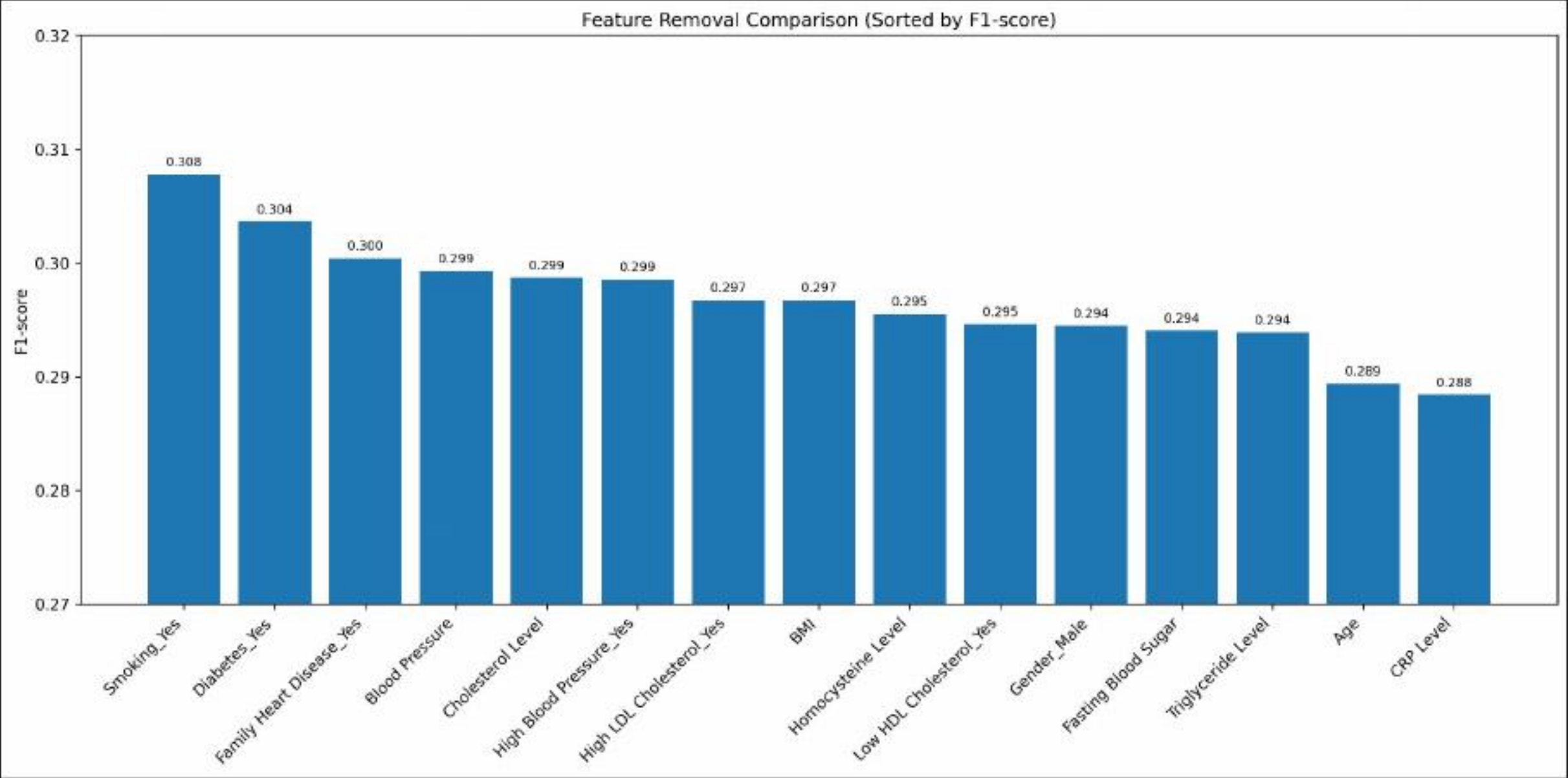
RANDOM FOREST

SGD

SVM

02

	Removed Feature	Accuracy	Precision (Class 1)	Recall (Class 1)	F1-Score (Class 1)
0	Age	0.3910	0.188737	0.6200	0.289382
1	Blood Pressure	0.5200	0.211340	0.5125	0.299270
2	Cholesterol Level	0.5210	0.211180	0.5100	0.298682
3	BMI	0.3885	0.192681	0.6450	0.296722
4	Triglyceride Level	0.3945	0.191635	0.6300	0.293878
5	Fasting Blood Sugar	0.3950	0.191781	0.6300	0.294049
6	CRP Level	0.3930	0.188361	0.6150	0.288394
7	Homocysteine Level	0.3825	0.191426	0.6475	0.295493
8	Gender_Male	0.4010	0.192604	0.6250	0.294464
9	Smoking_Yes	0.3725	0.197452	0.6975	0.307777
10	Family Heart Disease_Yes	0.3875	0.194671	0.6575	0.300400
11	Diabetes_Yes	0.3945	0.197162	0.6600	0.303623
12	High Blood Pressure_Yes	0.3985	0.194677	0.6400	0.298542
13	Low HDL Cholesterol_Yes	0.3870	0.191330	0.6400	0.294591
14	High LDL Cholesterol_Yes	0.3885	0.192681	0.6450	0.296722



TUNED MODELS

LOGISTIC
REGRESSION

DECISION TREE

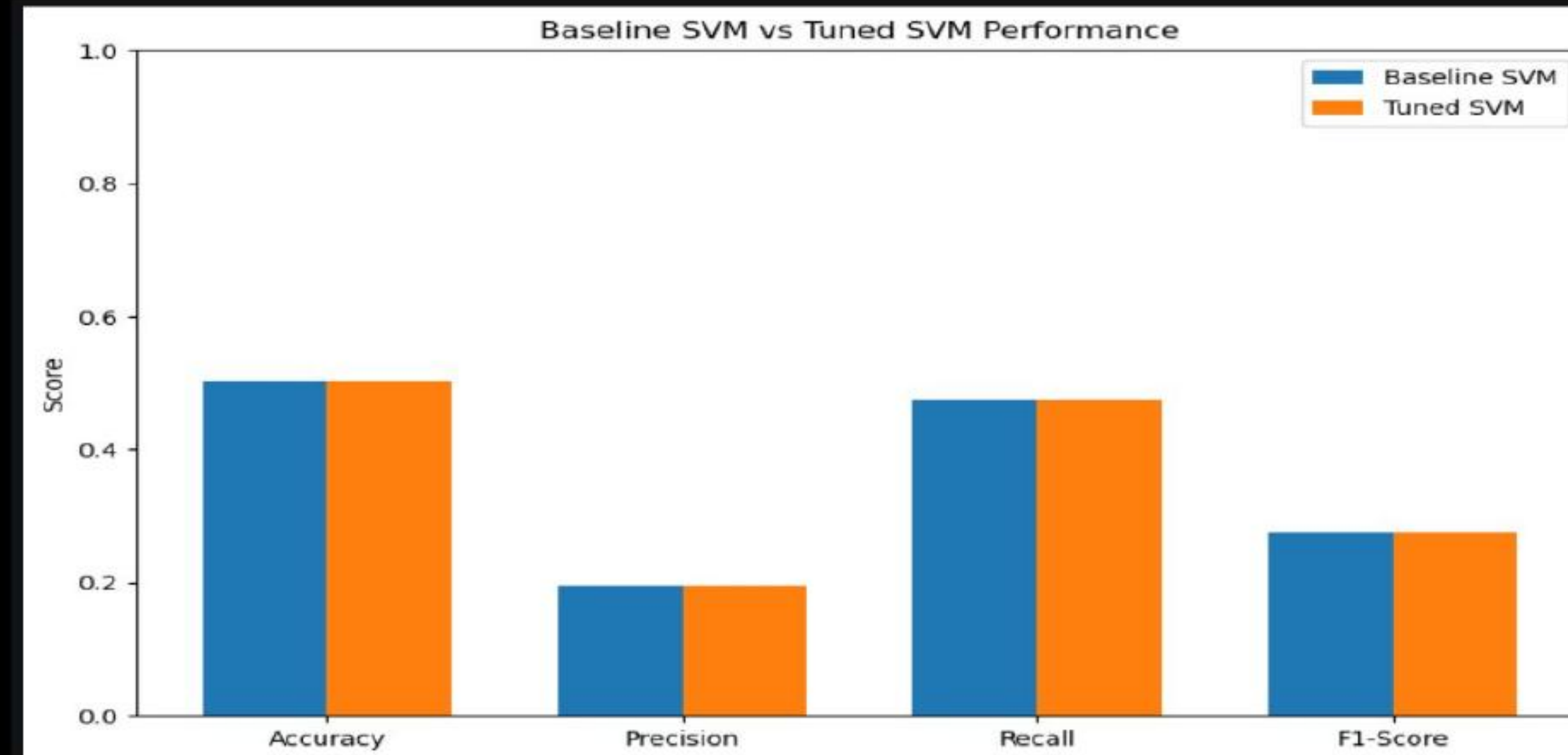
RANDOM FOREST

SGD

SVM

```
from sklearn.model_selection import GridSearchCV
param_grid = {
    'C': [0.1, 1, 10],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale']
}
svm = SVC(
    random_state=42,
    class_weight='balanced'
)
grid_search = GridSearchCV(
    svm,
    param_grid,
    cv=3,
    scoring='recall'
)
grid_search.fit(X_train, y_train)
print("Best Parameters:", grid_search.best_params_)
print("Best Recall Score (CV):", round(grid_search.best_score_, 4))
```

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
svm_model = SVC(
    kernel='rbf',
    random_state=42,
    class_weight='balanced'
)
svm_model.fit(X_train, y_train)
y_pred = svm_model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

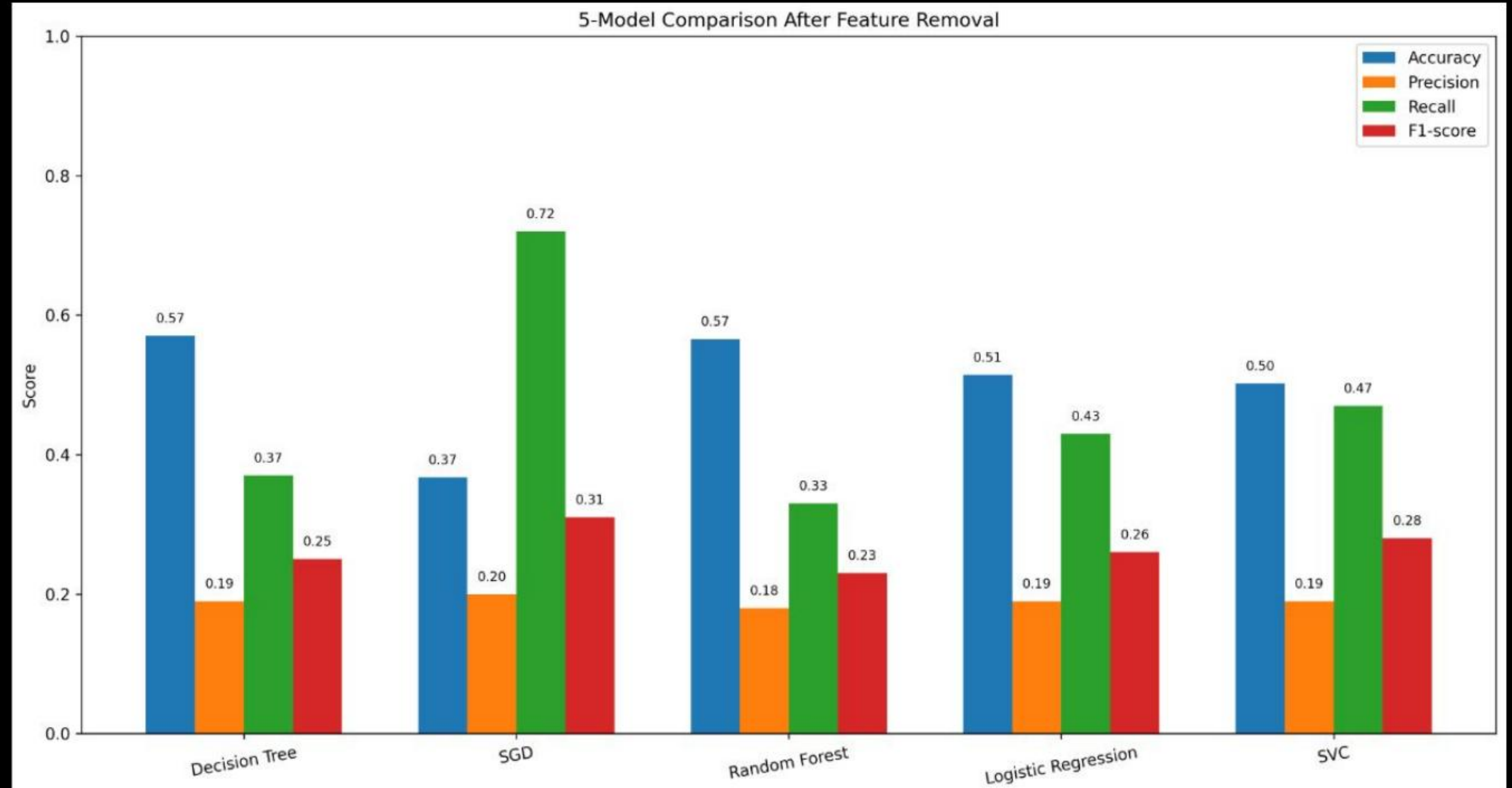


02

FEATURE ABLATION

REVIEW MODEL PERFORMANCE

Drop Random Feature: smoking, diabetes 🤖



🤖

03

NEXT STEPS

Which ML approach is the best for our disease classification problem?

Current Findings:

SGD model is giving us the best recall fit for our dataset.
SGD is good at large-scale, streaming EHR (electronic health record) data but requires extensive tuning and data preprocessing.
Opens up possibilities for future production scaling into a more complex disease detection algorithm.